Step 1 — The Help-Docs RAG and the Confidence Gate — Solution

> Back to the step · Hint 1 · Hint 2

Story: the help store, the retriever, and the deterministic confidence gate — the one decision the whole bot turns on, proven with no model. **If not:** if the gate is wrong, the bot either answers off-topic questions (confidently wrong) or escalates everything (useless).

Setup files

`requirements.txt`: `langchain-openai`, `langchain-community`, `chromadb`, `langgraph`, `langgraph-checkpoint-sqlite`, `openai`, `python-dotenv`. `.env.example`: `OPENAI_API_KEY=sk-your-key-here` / `LOG_LEVEL=INFO`. `.gitignore`: `.env`, `.venv/`, `__pycache__/`, `*.pyc`, `chroma_db/`, `*.sqlite`. Every folder under `src/` gets an empty `__init__.py`. `LICENSE` is the MIT text.

Settings, errors, logging, model

```
# src/exceptions.py
class MissingConfigError(Exception):
    """A required setting is missing from the environment."""
    pass
```

```
# src/config.py
import os
from dataclasses import dataclass
from dotenv import load_dotenv
from src.exceptions import MissingConfigError


@dataclass
class Config:
    openai_api_key: str
    log_level: str
    confidence_cutoff: float
```

```
def require_env(key: str) -> str:
    value = os.getenv(key)
    if value is None or value == "":
        raise MissingConfigError(
            f"Required environment variable is missing: {key}"
        )
    return value


def load_config() -> Config:
    load_dotenv()
    return Config(
        openai_api_key=require_env("OPENAI_API_KEY"),
        log_level=os.getenv("LOG_LEVEL", "INFO"),
        confidence_cutoff=float(os.getenv("CONFIDENCE_CUTOFF", "0.8")),
    )
```

```
# src/utils/logger.py
import logging
import os


def get_logger(name: str) -> logging.Logger:
    logger = logging.getLogger(name)
    if not logger.handlers:
        level_name = os.getenv("LOG_LEVEL", "INFO")
        level = getattr(logging, level_name.upper(), logging.INFO)
        handler = logging.StreamHandler()
        handler.setLevel(level)
        log_format = "%(asctime)s %(name)s %(levelname)s %(message)s"
        handler.setFormatter(logging.Formatter(log_format))
        logger.addHandler(handler)
        logger.setLevel(level)
    return logger
```

```
# src/models/llm.py
from langchain_openai import ChatOpenAI

from src.config import Config

MODEL_NAME = "gpt-4o-mini"


def get_model(config: Config) -> ChatOpenAI:
```

```
    return ChatOpenAI(
        model=MODEL_NAME, temperature=0, api_key=config.openai_api_key
    )
```

The help-docs RAG

```
# src/rag/help_store.py
# Load the help articles, embed them, and store them in Chroma so a question
# can be matched to the closest article by meaning (Doc08).
import os

from langchain_community.document_loaders import DirectoryLoader, TextLoader
from langchain_openai import OpenAIEmbeddings

import chromadb

COLLECTION_NAME = "help_docs"
HELP_DIR = "data/help_docs"
EMBEDDINGS = OpenAIEmbeddings(model="text-embedding-3-small")


def build_help_store(help_dir: str = HELP_DIR):
    loader = DirectoryLoader(help_dir, glob="**/*.md", loader_cls=TextLoader)
    documents = loader.load()
    texts = []
    for document in documents:
        texts.append(document.page_content)

    client = chromadb.Client()
    try:
        client.delete_collection(COLLECTION_NAME)
    except Exception:
        pass
    collection = client.create_collection(COLLECTION_NAME)

    ids = []
    for index in range(len(texts)):
        ids.append(f"doc-{index}")
    collection.add(ids=ids, documents=texts,
                   embeddings=EMBEDDINGS.embed_documents(texts))
    return collection
```

```
# src/rag/retriever.py
# Return the closest help article to a question, with its distance.
```

```
from src.rag.help_store import EMBEDDINGS

NO_MATCH_DISTANCE = 2.0


def best_match(collection, question: str):
    if collection.count() == 0:
        return "", NO_MATCH_DISTANCE
    results = collection.query(
        query_embeddings=[EMBEDDINGS.embed_query(question)], n_results=1,
    )
    documents = results["documents"][0]
    distances = results["distances"][0]
    if not documents:
        return "", NO_MATCH_DISTANCE
    return documents[0], distances[0]
```

The confidence gate (the one decision)

```
# src/confidence.py
# The whole bot turns on this: is the best match close enough to answer from?
# Deterministic, so it's testable with no model.
DEFAULT_CUTOFF = 0.8


def is_confident(distance: float, cutoff: float = DEFAULT_CUTOFF) -> bool:
    return distance <= cutoff
```

The tests (no model, no network)

```
# tests/unit/test_config.py
# Run from the repo root: python -m tests.unit.test_config
import os

from src.config import load_config, require_env
from src.exceptions import MissingConfigError

TEST_SETTING = "DEFLECTDESK_TEST_SETTING"


def test_missing_setting() -> None:
    os.environ.pop(TEST_SETTING, None)
```

```
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 1 (missing setting) -> {result}")


def test_config_loads() -> None:
    os.environ["OPENAI_API_KEY"] = "sk-test-123"
    config = load_config()
    result = "OK"
    if config.openai_api_key != "sk-test-123":
        result = "WRONG"
    print(f"case 2 (config loads)    -> {result}")


if __name__ == "__main__":
    test_missing_setting()
    test_config_loads()
```

```
# tests/unit/test_confidence.py
# Run from the repo root: python -m tests.unit.test_confidence
from src.confidence import is_confident


def test_small_distance_is_confident() -> None:
    result = "OK"
    if not is_confident(0.2, cutoff=0.8):
        result = "WRONG"
    print(f"case 1 (small distance -> confident)  -> {result}")


def test_large_distance_is_not() -> None:
    result = "OK"
    if is_confident(1.5, cutoff=0.8):
        result = "WRONG"
    print(f"case 2 (large distance -> escalate)   -> {result}")


def test_cutoff_itself_is_confident() -> None:
    result = "OK"
    if not is_confident(0.8, cutoff=0.8):
        result = "WRONG"
    print(f"case 3 (distance == cutoff -> confident) -> {result}")
```

```
if __name__ == "__main__":
    test_small_distance_is_confident()
    test_large_distance_is_not()
    test_cutoff_itself_is_confident()
```

Expected output:

```
case 1 (missing setting) -> OK
case 2 (config loads)    -> OK
```

```
case 1 (small distance -> confident)  -> OK
case 2 (large distance -> escalate)   -> OK
case 3 (distance == cutoff -> confident) -> OK
```

The gate is one tiny function — and it's the whole decision the bot makes. Proving it with made-up distances means Step 2's graph only has to route on it.